SpikeWorld: Adapting Frozen World Models with Compact State

Ziqiao Yu, Hongming Zhang, Kai Wang, Runze Wei, Runze Cui,
Yizhou Tan, Bowen Zhou, Siming Lu, Meijun Guo, Qingfeng Tang

November 2026

Abstract

World models can learn from their own prediction errors after deployment, but updating a shared model also changes the representation used by its other tasks. We introduce SpikeWorld, a predictive–semantic spiking model that keeps its trained parameters fixed and learns deployment corrections in compact external state. We train a shared causal core to predict future sensory features and action outcomes, together with semantic recognition and cross-modal association. At deployment, we use observed prediction errors to update residual matrices that refine future predictions and candidate scores that select bounded action corrections. On a six-stage stream of registered actuator changes, 28.8 KB of mutable numeric state reduces prediction error by 19.3%, compared with 21.9% for the evaluated prediction-path fine-tuning configuration using 1.93 MB. We test the components separately to identify which improves prediction and which changes actions. A closed-loop study shows that the action correction improves reward. Throughout adaptation, the trained model and its semantic outputs on fixed inputs remain unchanged. These results show that a predictive model can use deployment experience to improve its predictions while preserving its learned semantic mapping.

1 Introduction

Imagine a robot reaching for an object after its motors have become weaker. The same command now produces a different movement. A world model can detect this discrepancy: it predicts the outcome of a command, observes the actual outcome, and measures the difference. The question is how to use that new experience. Updating the model is one option. Another is to keep the trained model intact and learn a small correction alongside it.

This second option is useful when a representation serves several purposes. A model may predict future observations, recognize semantic categories, and associate information across sensors. Deployment feedback about an actuator provides a direct signal for dynamics prediction, while the model's other outputs may need to remain stable. A separate correction can learn from this feedback while the trained representation remains fixed.

World models learn representations through prediction (Ha & Schmidhuber, 2018; Hafner et al., 2019, 2020). For a sequence predictor, the timing and order of observations can matter as much as their content. We choose a spiking temporal core to make recent input history part of the computation: membrane variables accumulate incoming signals, leak over time, and emit spikes when their accumulated value crosses a threshold. This gives the model a mechanism for representing temporal patterns, which surrogate-gradient training can learn to use (Yu et al., 2026). Causal attention combines these signals using only the observed prefix. Our controlled event-model probes support this temporal motivation: future prediction is more

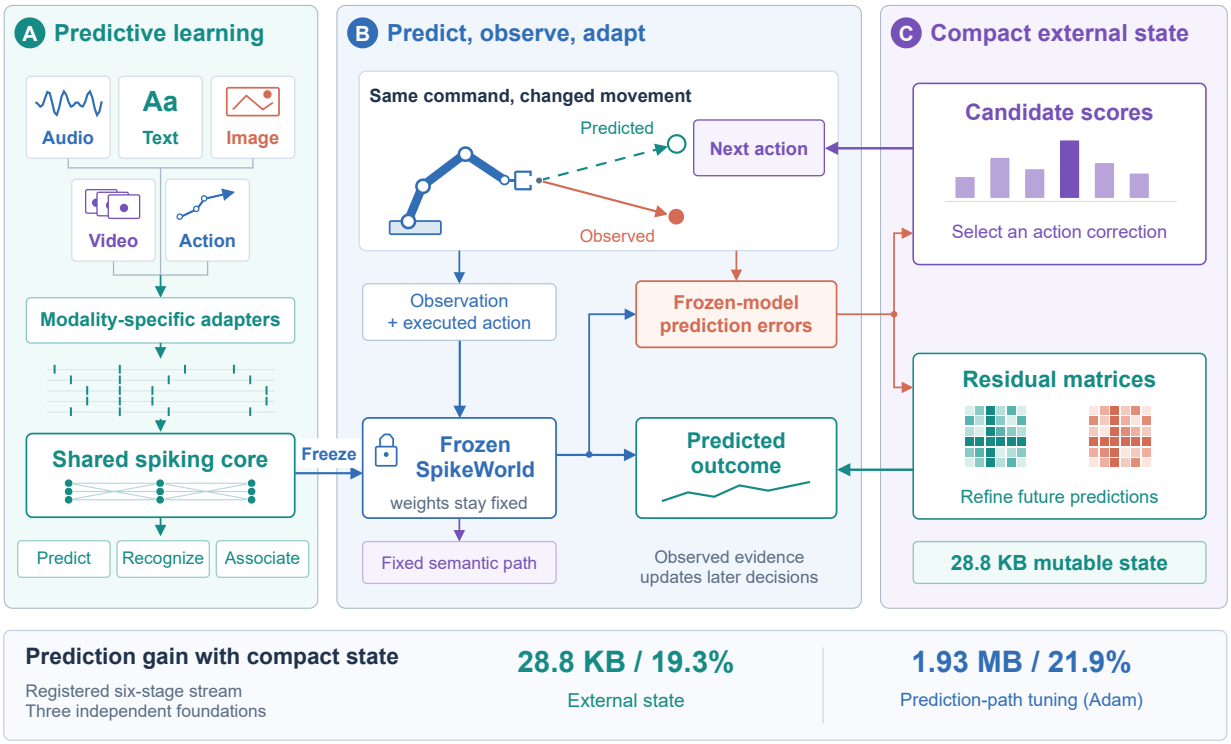


Figure 1: A shared predictive model, with room to learn outside its weights. A, sensory and action inputs enter through modality-specific adapters and train a shared spiking core for prediction, recognition, and association. **B**, the selected checkpoint is frozen. The robot illustration shows how changed actuation creates a difference between expected and observed movement. Outcomes evaluated through the frozen predictor supply online learning evidence. **C**, candidate scores select a bounded correction to a later command (purple); residual matrices refine the prediction (teal). The semantic path retains its fixed-input mapping. The bottom strip summarizes the main-stream prediction–storage comparison. Icons, trajectories, scores, and matrix patterns are schematic; the bottom-strip values are measured results.

accurate with ordered sequences than when event features are shuffled or sorted onto a regular time grid (Section 5.4).

Audio, visual features, rendered words, and state–action inputs enter through appropriate adapters. Each adapter preserves its stream's prediction target; all streams use the same temporal core. We use *next-spike prediction* to describe this event-oriented learning approach and specify the actual next-feature and action-outcome targets in Section 2.

Figure 1 connects this learning process to deployment. The left panel gathers heterogeneous inputs into a shared predictive model. The center panel returns to the weakened-motor example: the selected model stays fixed while newly observed outcomes provide evidence about the change. The predictor trained offline thus supplies the error signal for online learning. The right panel shows where that experience is stored. Residual matrices correct hidden features for prediction, while a separate score vector compares registered actuator transformations and chooses a bounded command correction. An existing control policy proposes the command; this interface adjusts it to compensate for the identified actuator change. The colored paths show how each correction is used. We can disable either component to test its effect on prediction and commands.

We find that a small external state recovers most of the prediction improvement of the evaluated prediction-path fine-tuning configuration. This comparison uses a six-stage stream containing source, attenuation, and shear conditions and measures how much mutable state each method requires. Component interventions show that the matrices account for additional prediction accuracy, while the score-and-inverse path accounts

for changed commands. A separate closed-loop experiment connects the latter to reward.

We make three contributions. We train and evaluate a shared predictive–semantic spiking model with sensory and action targets. We design an online interface that stores corrections in bounded external state and preserves the trained semantic mapping. We measure its prediction–storage trade-off and test the separate roles of its state components. The appendix provides the full equations, protocols, additional comparisons, and code-to-result map. Code, editable figures, and recorded experiment outputs are available at https://github.com/Oooorca/SpikeWorld.

2 A shared model for prediction and semantics

We train the model to predict future observations and recognize categories from a shared feature sequence. Figure 2 shows the training tasks. Panel A predicts the next sensory slice from the observed prefix. Panel B predicts the later outcome of an action. Panel C shows how membrane accumulation and spikes process sequences, and which losses train the readouts.

2.1 Inputs, features, and spikes

Let m identify a sensory stream and let $x^m_{1:T} \in \mathbb{R}^{T \times D_m}$ contain T feature slices with D_m channels per slice. A slice is the numerical feature vector presented at one sequence position. The implementation uses $T = 16$, with modality-dependent channel counts. Learned input adapters and modality embeddings map these features into a shared width-d representation, where $d = 96$. The audio, image, text, and video adapters and their data sources are described in Appendix A.

A spiking neuron retains a membrane variable between sequence positions. Figure 2C illustrates how incoming information accumulates until the membrane crosses a threshold and emits a spike. For sensory position τ, its update is

$$v_\tau = \beta u_{\tau-1} + I_\tau, \qquad s_\tau = \mathbf{1}[v_\tau > V_{\mathrm{th}}], \qquad u_\tau = v_\tau - V_{\mathrm{th}} s_\tau. \tag{1}$$

Here I_τ is the incoming current, $u_{\tau-1}$ is the previous post-reset membrane value, v_τ is the value before reset, and s_τ is a binary spike. The indicator $\mathbf{1}[\cdot]$ is one when its condition holds and zero otherwise. The decay $\beta = 0.85$ retains part of the preceding value, and the threshold $V_{\mathrm{th}} = 1$ determines whether a spike is emitted. The last operation subtracts the threshold after a spike. During offline training, surrogate derivatives supply a gradient through the threshold (Neftci et al., 2019; Yu et al., 2026).

Equation (1) explains the temporal role of the spiking model. An input can affect later membrane values even if it does not immediately produce a spike. Decay gives more recent inputs greater weight, so rearranging the same inputs can change when the threshold is crossed. The next-slice objective trains the core to use this history to predict what follows. Membrane accumulation supplies local temporal context; causal attention combines information across the available sequence positions.

The shared core has two Transformer blocks and four attention heads. Each query attends to at most four causal positions, including itself. Spikes enter the attention and feed-forward paths, while residual connections carry numerical features. The learned representation is a numerical feature sequence. This implementation evaluates finite input sequences on conventional hardware; its sparse attention is specified in Appendix A.

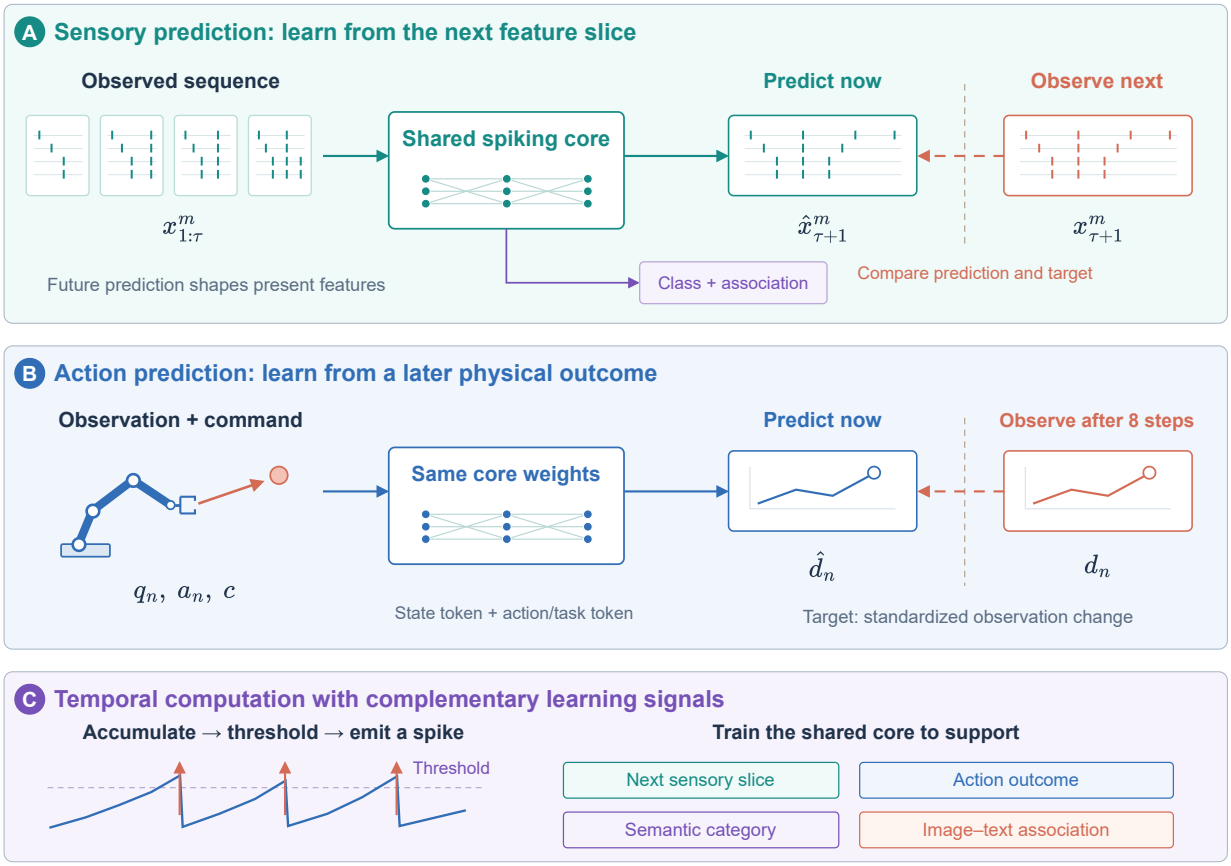


Figure 2: Predict the next sensory slice and the later consequence of an action. A, the observed sensory prefix produces a next-slice prediction; the next observation supplies its target. The same features also support class recognition and association. **B**, current observation, command, and task enter the same core to predict a standardized observation change, whose target becomes available after eight environment steps. Dashed vertical lines separate prediction time from target availability; coral arrows indicate target comparison. **C**, membrane accumulation and thresholded spikes provide temporal computation, while four objectives train complementary readouts of the shared core. Sensory position τ and action decision n use different clocks. Feature patterns, trajectories, and membrane traces are schematic illustrations.

2.2 Predicting the next sensory slice

In Figure 2A, the modality-specific output head uses positions 1 through τ to produce $\hat{x}^m_{\tau+1}$, a prediction of the next feature slice. The hat marks a prediction; $x^m_{\tau+1}$ without the hat is the observed target. The prediction loss for stream m is

$$\ell_m = \frac{1}{(T-1)D_m\, p_m} \sum_{\tau=1}^{T-1} \|\hat{x}^m_{\tau+1} - x^m_{\tau+1}\|_2^2. \tag{2}$$

The squared norm $\|\cdot\|_2^2$ sums squared errors across feature channels. The outer sum then adds these errors over the available next-slice pairs. Dividing by $(T-1)D_m$ averages over positions and channels. The positive constant p_m is the training-set error of the persistence baseline, which predicts the next slice by copying the current one. This normalization puts modalities with different numerical scales on comparable footing. The sensory loss $\mathcal{L}_{\text{next}}$ gives equal weight to audio, text, image, and video groups. The audio group first averages its two sources, SHD and SSC; Appendix A.3 gives the combined expression.

The core processes spikes, while the output adapters predict encoded feature slices in each stream's

feature space. The targets can therefore contain non-binary values. Here, next-spike refers to causal prediction over these event-oriented sensory sequences.

2.3 Predicting action outcomes

Figure 2B uses a different horizon: the consequence of executing a command. Use n for an action decision and t_n for its starting environment step. An observation $q_n = q(t_n) \in \mathbb{R}^{D_q}$ has $D_q = 39$ coordinates and contains the simulator's numerical observation at the start of that decision. A command $a_n \in [-1, 1]^4$ and task identifier c are supplied to the action branch. Let $q_n^+ = q(t_n + H)$ be the observed outcome after horizon $H = 8$ environment steps. The superscript $+$ marks this later observation. We predict how the observation changes over the interval, using the standardized target

$$d_n = (q_n^+ - q_n - \mu_\Delta) \oslash \sigma_\Delta, \qquad \mathcal{L}_{\text{act}} = \frac{1}{D_q} \| g_\theta(q_n, a_n, c) - d_n \|_2^2. \tag{3}$$

Here g_θ is the action-conditioned predictor with trained parameters θ. The vectors $\mu_\Delta, \sigma_\Delta \in \mathbb{R}^{D_q}$ are the mean and scale of training observation changes; $\oslash$ is coordinate-wise division. Thus d_n is a vector of observed changes, distinct from the scalar hidden width d introduced above. The factor $1/D_q$ averages error over output coordinates. In the implementation it is $1/39$, a dimensional normalization rather than a tuned coefficient. The state and action/task adapters provide two tokens to the shared core; the frozen linear endpoint head maps the final action feature to D_q outputs.

2.4 Learning predictions and semantic readouts together

The four losses in Figure 2C train prediction, recognition, and association from the same core. The coupled semantic heads read its features to predict category logits, the class scores before conversion to probabilities. Their cross-entropy loss is $\mathcal{L}_{\text{sem}}$. A binding head maps image and rendered-text features to normalized vectors, and a symmetric contrastive loss $\mathcal{L}_{\text{bind}}$ associates examples with the same digit identity. Joint training minimizes

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{next}} + \mathcal{L}_{\text{sem}} + \lambda_b \mathcal{L}_{\text{bind}} + \lambda_a \mathcal{L}_{\text{act}}, \qquad \lambda_b = 0.25, \quad \lambda_a = 4. \tag{4}$$

The fixed coefficients λ_b and λ_a set the relative weights of association and action prediction in the training loss. Equation (4) trains the core to preserve information useful for both future prediction and present classification. Appendix A.3 expands the classification and contrastive terms.

We evaluate two training stages separately. The *sequential checkpoint* begins with sensory pretraining and then trains the action path for 800 updates while holding the core fixed. This is the checkpoint used in the main deployment comparison. The *joint checkpoint* adds 200 shared-core updates with Equation (4). A separate study compares these stages to measure the effect of joint learning. Once a checkpoint is selected for deployment, all of its trained parameters are frozen.

3 Learning corrections outside the frozen model

Consider again the weakened motor. We compare the model's predicted outcome with the outcome observed in the environment, then use the difference to update two small objects. A score vector identifies a suitable command correction; a residual matrix improves the predicted outcome. The following description uses the decision index n and target d_n defined in Equation (3).

3.1 Predict first, update after observation

Let θ_0 denote the predictor parameters from the checkpoint selected for deployment. These parameters, the semantic heads, the mechanism router, and the underlying control policy are frozen. The router is an offline-trained classifier that chooses among source (unchanged actuation), shear, attenuation, and noise conditions. The online interface receives the current observation, executed command, task identifier, and eventual outcome. Reward, success, and the true actuator change are evaluation variables, not online update inputs.

Every prediction is recorded before its target is observed. After the horizon has elapsed, the system computes an error and updates external state for later decisions. The first four transitions in an episode supply evidence to the frozen mechanism router. The route locks after the fourth outcome arrives. The system can then update the persistent state for the first time. Consequently, decision five is the earliest command that can change. Appendix B gives the complete schedule, including the different lifetimes of episode prefixes and persistent matrices.

3.2 Scores select an actuator correction

The model compares a bank of $J = 32$ transformations: identity, shear, and attenuation candidates. Identity leaves the command unchanged. Shear couples the first two action coordinates, so that an x command also affects y. Attenuation reduces the realized movement in the x/y coordinates. The bank's values are registered offline; the online system infers a candidate from observed transitions.

For candidate j, let T_j be its command transformation. Once outcome d_n is available, the candidate receives loss

$$\ell_{n,j} = \frac{1}{D_q}\|g_{\theta_0}(q_n, T_j(a_n), c) - d_n\|_2^2, \qquad S_j \leftarrow S_j + \ell_{n,j}. \tag{5}$$

Here $\ell_{n,j}$ is the error for candidate j on the observed transition at decision n. Its cumulative score S_j starts at zero and adds the losses admitted by the routing schedule. A lower loss means that the candidate transformation better explains the observed transition through the frozen predictor. A frozen router first selects the mechanism family ρ from the four initial transitions; within that family, the candidate with the smallest S_j supplies the estimated transformation. These comparisons always query the original frozen predictor, without the residual matrix described below. Source and noise routes keep the command unchanged and skip matrix updates.

Let a_n^* be the command proposed by the underlying policy, and let $\tilde{a}_n$ be the selected inverse transformation applied to that command. The controller executes

$$a_n = a_n^* + \mathrm{clip}(\tilde{a}_n - a_n^*, -\delta, \delta), \qquad \delta = 0.05. \tag{6}$$

The star marks the policy's desired command; the tilde marks the inverse's proposed command. The final command a_n limits how far the proposal can move from the desired command. The clip acts separately on each coordinate, with δ setting the maximum change in either direction. If this capped command would lie outside the allowed action range, the system uses a_n^* instead. The inverse maps, gain floor, and source/noise behavior are given in Appendix B.

For a one-coordinate illustration, suppose the route has locked and prior observations have selected gain 0.5. A desired command of 0.10 gives inverse proposal 0.20. The 0.05 cap makes the executed command 0.15. If the actual gain is 0.5, its realized effect is 0.075, compared with 0.05 without correction. Candidate scoring uses the forward gain transform to explain observed outcomes; the inverse compensates a later command. The residual matrix below changes the predicted outcome, not this command. These illustrative values explain the operation rather than report an additional experiment.

3.3 Matrices refine the prediction

After selecting a candidate, the frozen core computes hidden feature $h_n \in \mathbb{R}^d$ using the current observation, task, and transformed command. This is an internal feature vector computed for the current input. We retain the fixed offline mean $\bar{h} \in \mathbb{R}^d$ and projection $B \in \mathbb{R}^{r \times d}$, where $r = 30$. The projection is obtained by principal component analysis of offline action features. Each correctable mechanism family has its own matrix $C_\rho \in \mathbb{R}^{d \times r}$, initialized to zero. Prediction becomes

$$z_n = B(h_n - \bar{h}), \qquad \tilde{h}_n = h_n + C_\rho z_n, \qquad \hat{d}_n = W_{\text{out}} \tilde{h}_n + b_{\text{out}}. \tag{7}$$

The vector $z_n \in \mathbb{R}^r$ compresses the centered feature. Here r is the number of retained feature directions, and ρ selects the shear or attenuation matrix. Multiplication by C_ρ expands the compressed feature into a learned correction. The tilde in $\tilde{h}_n$ marks the corrected hidden feature, and $\hat{d}_n$ is its predicted observation-change vector. The original readout has weight $W_{\text{out}} \in \mathbb{R}^{D_q \times d}$ and bias $b_{\text{out}} \in \mathbb{R}^{D_q}$; both remain frozen. The prediction changes through the residual matrix; the feature extractor and readout keep their trained weights. The features h_n and z_n are computed anew for each input; C_ρ stores the correction learned across observed transitions.

After observing an outcome and updating the candidate scores, the active matrix takes a normalized gradient step on prediction mean squared error (MSE). The step length is $\eta = 0.02$, and the matrix is projected to a Frobenius-norm ball of radius $R_C = 0.20$. In this update, η controls the step length after gradient normalization. The Frobenius norm measures the overall matrix size as the square root of the sum of its squared entries. If an update exceeds R_C, we scale the matrix back to this bound. Appendix B.3 spells out this update and explains how it differs from the earlier pre-update error used for evaluation. Candidate scoring and the action inverse do not read the matrices. We can therefore remove the matrices to measure their prediction effect while leaving action selection unchanged.

3.4 What remains fixed, and what occupies memory?

The trained semantic computation has no read connection to the external matrices or scores. Given the same sensory input and inference settings, it therefore returns the same logits (pre-softmax class scores) before and after online adaptation. We verify this with parameter and fixed-input-output hashes. This property preserves the learned mapping; semantic outputs can still differ when the robot observes different inputs.

The online allocation contains two residual matrices, the score vector and count, compatibility padding, and a four-transition routing prefix. Their peak total is 28,848 bytes of mutable numeric state. This is the memory that can change during adaptation, separate from the frozen model and transient inference activations. Table 6 gives the exact arithmetic. The bounded prefix is cleared after routing, and the matrices and scores retain experience across the stream.

4 Evaluation design

We test whether the shared core learns predictive and semantic features, how much mutable state online adaptation requires, and which component changes behavior. The studies use different experimental units, listed with their released records in Appendix C.1.

Representation and joint learning. The sensory sources include SHD/SSC audio (Cramer et al., 2022), retinal digit images and rendered words, and a causal video-feature cache. Action sequences use Meta-World

reach, push, pick-place, and door-open tasks (Yu et al., 2020). We call a sensory-pretrained checkpoint a *foundation*; an action seed specifies a fresh initialization of its action branch. Five action initializations sharing one sensory foundation compare sequential training, action-only updating, and joint training. Three additional controlled event-model studies examine temporal structure, prediction of future observation statistics, and image/audio factor association. They use separate probe models and three seeds each. These probe models are separate from the checkpoints used in the deployment comparison.

The main deployment stream. Three independently trained foundations, each with three action seeds, produce nine paired runs. Each uses the sequential checkpoint. All methods see the same six-stage layout: source, attenuation, source, shear, attenuation, and source return. Every stage includes the four manipulation tasks. The tested shift values belong to the registered bank. Shifted trajectories come from an execution archive separate from router calibration. Source stages revisit calibration episodes, so source-return measurements quantify calibrated reentry.

The five configurations are frozen prediction, external state, action-adapter tuning, prediction-path tuning, and prediction-path tuning with replay. All three trainable comparison arms use Adam. "Prediction-path tuning" refers to the deployed prediction path, not every parameter in the multimodal model. We count mutable parameters, persistent optimizer tensors, state and routing buffers, and replay where applicable. We also record update latency using CUDA-synchronized measurements after warm-up.

The main study replays a common recorded trajectory in causal order: each prediction is evaluated before its outcome is used to update the method. Each run contains 348 transitions, of which 174 are shifted. The primary score uses the 126 shifted transitions remaining after excluding the first four transitions of every episode, which establish its route. The score is the relative reduction in prediction MSE. Let M identify the evaluated method. Let E_{fr} and E_M be the mean prediction errors of the frozen predictor and method M on the same evaluated transitions. Their comparison gives prediction gain G_M:

$$G_M = 100 \left(1 - \frac{E_M}{E_{\mathrm{fr}}} \right) \%. \tag{8}$$

A positive gain means smaller error. Action tracking applies the known actuator map to each corrected command and measures its distance from the desired command. Replay gives every method the same observations for the prediction comparison. In the closed-loop test below, corrected commands change subsequent observations, and reward measures the resulting behavior. Source-return correction measures the size of unnecessary command adjustments upon calibrated reentry. Action seeds are averaged within each foundation. Confidence intervals (CIs) use 20,000 bootstrap draws over the three foundation means, preserving paired method comparisons.

Component and closed-loop tests. The component study compares full state, score-only state, and removal of the inverse. A separate 36-trace check verifies that bounding the routing history preserves the original implementation's observable behavior. Closed-loop attenuation uses five model seeds, three tasks, and five environment replicas: 75 trajectories per method. The tasks are reach, push, and pick-place, with attenuation gain 0.4. Each trajectory contains 13 decisions over 104 environment steps. Adaptation state carries across the five replicas within each model/task pair; a new controller is created for the next pair. Reward and success are evaluation outputs, never adaptation inputs. Its CIs use paired resampling of model, task, and replica axes. The live-control tuning and replay controllers are defined separately in Appendix C.7.

Classical calibration and training-stage diagnostics. A matched six-stage replay compares the external state with routed actuator-space recursive least squares (RLS), using a two-by-two estimate for action

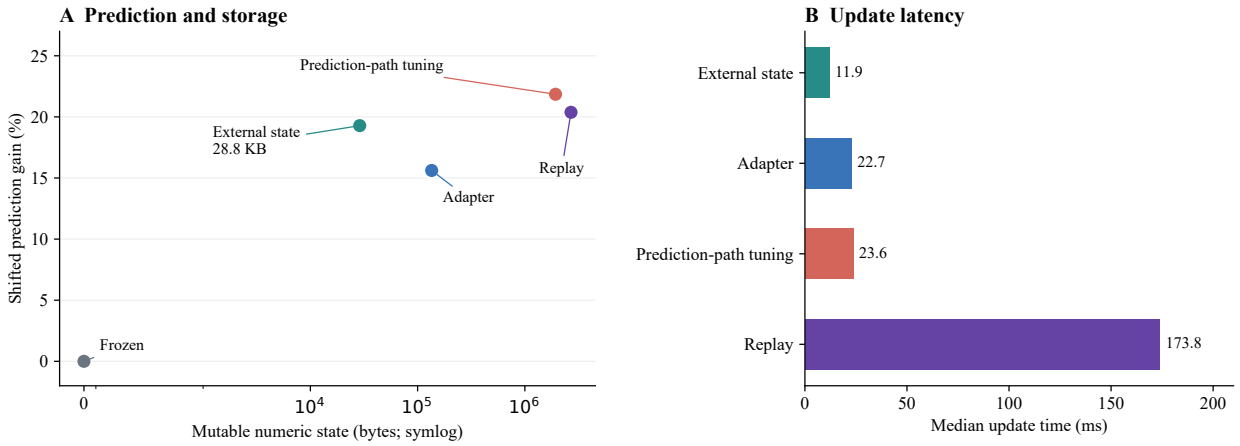


Figure 3: External state provides a compact deployment configuration. Left: prediction gain against peak mutable numeric storage for the five main arms. Labels identify every point; the horizontal axis uses a symmetric-log scale to include frozen prediction at zero bytes. Right: median update latency for the adaptive arms. All arms use the same three-foundation, three-action-seed design. These are configuration-specific memory and latency measurements.

Table 1: Main six-stage replay comparison. Gains use 126 shifted transitions per run after routing prefixes; all trainable arms use Adam. Mutable state excludes frozen weights and transient activations.

Method	Pred. gain (%)	Tracking (%)	Bytes	Update (ms)
Frozen	0.00	0.00	0	–
External state	19.28	3.80	28,848	11.90
Adapter (Adam)	15.61	3.47	135,232	22.72
Prediction-path tuning (Adam)	21.85	2.91	1,932,488	23.62
Replay (Adam)	20.38	3.35	2,693,128	173.83

coupling. The closed-loop study includes a separate scalar RLS reference for attenuation. Appendix C.6 also reports a one-foundation development study of direct output-space calibration, including normalized least mean squares (NLMS), RLS, and a low-rank readout update. We report each comparison under its own protocol because the calibrators correct different quantities. A further one-foundation pilot applies the same external-state interface to different offline training stages.

5 Results

5.1 A compact prediction–storage trade-off

External state improves shifted prediction MSE by 19.28% with 28,848 mutable bytes. Prediction-path tuning reaches 21.85% with 1,932,488 bytes under the evaluated Adam configuration. The difference in gain is 2.57 percentage points; the difference in mutable storage is about 67-fold. Figure 3 and Table 1 place this result beside the other evaluated configurations. The external-state point is non-dominated among these five configurations: no other point has both less mutable storage and greater mean prediction gain.

The external-state mean prediction gain has a foundation-bootstrap 95% CI of 15.71%–21.80%. Its mean tracking gain is 3.80%, and calibrated-source return produces zero command correction. Median update time is 11.9 ms, approximately half that of the adapter and prediction-path tuning arms. External state thus

Table 2: Component intervention. Full and score-only commands are bitwise equal; matrix contribution is their difference in prediction gain.

Shift	Full gain (%)	Scores (%)	Matrix (points)	Tracking (%)
Shear	8.13	-1.09	9.22	43.99
Attenuation	32.63	7.51	25.12	3.88

Table 3: Closed-loop attenuation: 75 trajectories per arm; adaptation carries across replicas within each model/task pair. Reward and success are evaluation-only. Controller definitions appear in Appendix C.7.

Method	Mean reward	Success (%)
Frozen	414.68	53.33
Prediction-model tuning (AdamW)	422.19	66.67
Oracle inverse (uncapped correction)	485.50	80.00
Gain-memory replay	428.59	60.00
Scalar RLS	434.41	65.33
External state	422.58	66.67

improves prediction with a small allocation and a short update time in this comparison. Appendix C reports the values for each foundation.

In the separate matched actuator-identification replay, external state obtains 19.72% prediction gain and the routed matrix RLS obtains 0.58%. That comparison isolates actuator-space identification on the six-stage layout. In the separate development protocol in Appendix C.6, direct output-space calibrators achieve larger prediction gains with less state. Where the correction is applied therefore matters to this comparison.

5.2 Prediction and action gains come from different components

Table 2 makes the role of the residual matrices explicit. Relative to scores alone, matrices add 9.22 percentage points of prediction gain under shear and 25.12 points under attenuation. Yet the full and score-only variants execute exactly the same commands in the paired intervention. Removing the inverse makes action improvement zero. The matrices improve predictions; the scores and inverse change commands. The interventions separate these two effects of online adaptation.

The bounded-memory implementation preserves this behavior. Across the 36 traces in nine matched four-controller scenarios, the original and bounded versions agree in observable fields, and the first changed command occurs at decision five. This verifies that the small runtime footprint does not depend on retaining an expanding transition history. Throughout the main deployment runs, parameter hashes and fixed-input semantic-logit hashes remain unchanged.

5.3 The action correction improves closed-loop reward

In registered attenuation, external state raises mean cumulative reward from 414.68 to 422.58. The paired reward improvement is 7.90, with a 95% CI from 2.48 to 14.06. Figure 4 shows how the reward difference develops after the observation prefix; Table 3 includes all closed-loop comparison arms. The reward gain is positive on each of the three tasks and across all five model seeds. These results show that the score-and-inverse path improves behavior in this attenuation study, alongside the matrices' separate effect on prediction.

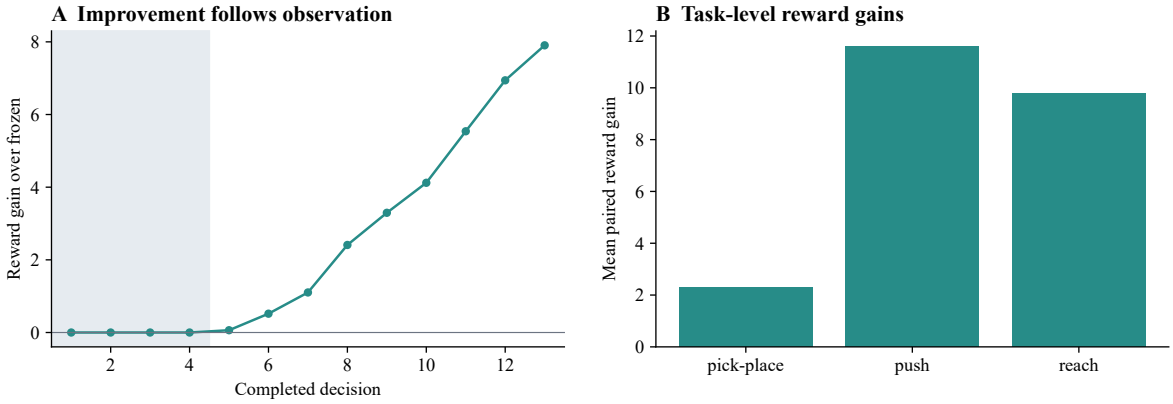


Figure 4: Behavior changes after evidence is available. Left: mean cumulative reward difference between external state and frozen control, plotted at completed decisions. The shaded prefix contains the first four decisions; decision five is the earliest corrected command. Right: paired reward gains by task. The separate attenuation study uses 75 trajectories per arm. Reward is used for evaluation, not for updating external state.

Table 4: Joint training relative to the sequential checkpoint. Five action initializations share one sensory foundation. Intervals are paired 95% bootstrap intervals.

Metric	Mean improvement	95% CI
Action MSE reduction	17.10%	[14.04, 20.04]%
Sensory MSE reduction	0.75%	[0.68, 0.83]%
Semantic accuracy change	1.80 points	[1.22, 2.44] points
Image–text retrieval change	1.81 points	[0.19, 3.44] points

5.4 Shared predictive training supports several readouts

Relative to the sequential checkpoint, joint training reduces action MSE by 17.10% and normalized sensory prediction error by 0.75%. Semantic accuracy and image–text class retrieval rise by 1.80 and 1.81 percentage points, respectively. These paired means use five action initializations sharing a sensory foundation. Table 4 reports their intervals. These changes show that joint updates improve both predictive and semantic readouts relative to the sequential checkpoint. The complete three-stage comparison in Appendix C.4 also shows the trade-off with action-only updates: joint training improves class retrieval, while action-only updating achieves the lower action error.

Temporal structure supports future prediction. In a separate controlled event-model study, ordered sequences yield a mean negative log-likelihood (NLL; lower is better) of 4.91, compared with 6.93 for the rate-grid control and 11.04 for shuffled order, across three seeds. The rate-grid control sorts event features and assigns evenly spaced timestamps; the order control shuffles non-time event features across fixed timestamps. This comparison shows that temporal organization carries useful predictive information in the probe task, supporting the motivation for a core that processes input history. Appendix C.3 gives the probe protocol and full results. These probe encoders are trained separately from the deployment checkpoints.

Two further probes test future observation statistics and image/audio factor associations. The training-stage pilot in Appendix C.5 then connects different trained checkpoints to the same external-state interface. The probes test what the representation learns; the pilot tests how the external correction behaves on checkpoints from different training stages.

6 Related work

World models and predictive state. World models learn representations that support forecasting and control (Ha & Schmidhuber, 2018; Hafner et al., 2019, 2020). Predictive state representations likewise describe a system through predictions of future observations (Littman et al., 2001). SpikeWorld uses prediction in two phases: offline to train sensory and action features, and online to obtain outcome-based correction signals. It stores those corrections outside the trained core.

Spiking temporal representations. Surrogate gradients enable optimization through discrete spike thresholds (Neftci et al., 2019; Zenke & Ganguli, 2018), and can support learning from spike timing (Yu et al., 2026). Spiking Transformers combine spike-based processing with attention (Zhou et al., 2023). Event-based sensing provides temporal contrast streams (Gallego et al., 2022); our experiments also use encoded feature sequences from conventional data sources. We study how the shared core predicts these sequences and specify each stream's input encoding and target.

Online adaptation and compact memory. Test-time training uses self-supervised deployment signals to update a model (Sun et al., 2020). Fast weights and differentiable plasticity separate rapidly changing information from slower learned parameters (Ba et al., 2016; Miconi et al., 2018). SpikeWorld implements a related separation through a fixed-size residual interface and a separate candidate score state. The model's original semantic computation remains outside these online writes.

Adaptive control. Online system identification and multiple-model adaptive control estimate changes and adjust commands (Narendra & Balakrishnan, 1997; Yu et al., 2017). Learned motor adaptation can infer context across changing robot dynamics (Kumar et al., 2021). SpikeWorld's candidate-score path belongs to this identification-and-correction setting. Its residual matrices add a distinct learned prediction correction, whose contribution is isolated experimentally. The appendix compares corrections applied in actuator space and at the prediction output.

7 Discussion and conclusion

We asked where a world model should store what it learns after deployment. SpikeWorld uses the difference between predicted and observed outcomes to learn compact external corrections. The trained predictive–semantic model remains fixed, so its semantic outputs on the same inputs remain unchanged. This preservation follows from the computation itself and requires no balancing of online semantic and adaptation losses.

Residual matrices refine predictions; candidate scores select a registered command correction. Removing each component in turn identifies its effect, and the closed-loop study shows that action correction improves reward. These results suggest that a deployment memory should be described by both the evidence it stores and the computation that reads it. Components trained from the same observed error can affect prediction and action differently.

The measured configuration improves prediction with a small mutable state while preserving the original model. Its shared temporal core supports sensory and action prediction together with semantic readouts. We release the model and adaptation code, comparison protocols, and recorded results so that the predictive core and online correction can be studied independently.

References

Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://proceedings.neurips.cc/paper/2016/hash/9f44e956e3a2b7b5598c625fcc802c36-Abstract.html.

Benjamin Cramer, Yannik Stradmann, Johannes Schemmel, and Friedemann Zenke. The heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 33(7):2744–2757, 2022. doi: 10.1109/TNNLS.2020.3044364.

Guillermo Gallego, Tobi Delbruck, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, Andrew J. Davison, Jorg Conradt, Kostas Daniilidis, and Davide Scaramuzza. Event-based vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(1):154–180, 2022. doi: 10.1109/TPAMI.2020.3008413.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/2de5d16682c3c35007e4e92982f1a2ba-Abstract.html.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2555–2565. PMLR, 2019. URL https://proceedings.mlr.press/v97/hafner19a.html.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=S1lOTC4tDS.

Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Robotics: Science and Systems XVII*, 2021. doi: 10.15607/RSS.2021.XVII.011.

Michael L. Littman, Richard S. Sutton, and Satinder Singh. Predictive representations of state. In *Advances in Neural Information Processing Systems*, volume 14, pp. 1555–1561, 2001.

Thomas Miconi, Kenneth O. Stanley, and Jeff Clune. Differentiable plasticity: Training plastic neural networks with backpropagation. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 3559–3568. PMLR, 2018. URL https://proceedings.mlr.press/v80/miconi18a.html.

Kumpati S. Narendra and Jeyendran Balakrishnan. Adaptive control using multiple models. *IEEE Transactions on Automatic Control*, 42(2):171–187, 1997. doi: 10.1109/9.554398.

Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019. doi: 10.1109/MSP.2019.2931595.

Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 9229–9248. PMLR, 2020. URL https://proceedings.mlr.press/v119/sun20b.html.

Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pp. 1094–1100. PMLR, 2020. URL https://proceedings.mlr.press/v100/yu20a.html.

Wenhao Yu, Jie Tan, C. Karen Liu, and Greg Turk. Preparing for the unknown: Learning a universal policy with online system identification. In *Robotics: Science and Systems XIII*, 2017. doi: 10.15607/RSS.2017.XIII.048.

Ziqiao Yu, Pengfei Sun, and Dan F. M. Goodman. Beyond rate coding: Surrogate gradients enable spike timing learning in spiking neural networks. *Neuromorphic Computing and Engineering*, 6:014016, 2026. doi: 10.1088/2634-4386/ae46d5.

Friedemann Zenke and Surya Ganguli. SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30(6):1514–1541, 2018. doi: 10.1162/NECO_A_01086.

Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=frE4fUwz_h.

A Notation, architecture, and training

This appendix defines the architecture, training objectives, and notation. The sensory sequence clock is indexed by τ; the robot's macro-decision clock is indexed by n. One macro-decision starts at environment step t_n and has nominal horizon $H = 8$. In the closed-loop runner, the transformed command is held for that interval, unless the episode terminates or reaches its remaining-step limit.

A.1 Notation at a glance

Table 5: Principal symbols. Scalars, dimensions, learned quantities, and fixed offline quantities are identified separately.

Symbol	Shape or value	Meaning
m, τ, T, D_m	Stream; position; 16; stream-specific	Sensory stream, sequence index, length, and channel count.
$x_\tau^m, \hat{x}_{\tau+1}^m$	$\mathbb{R}^{D_m}$	Observed current and predicted next feature slice.
p_m	Positive scalar	Persistence MSE used to normalize the sensory objective.
I_τ, v_τ, u_τ	Per-neuron values	Input current, pre-reset membrane, and post-reset membrane.
$s_\tau, \beta, V_{\text{th}}$	Binary; 0.85; 1	Spike, decay, and firing threshold.
n, t_n, H	Decision; environment step; 8	Separate macro-decision and simulator clocks.
q_n, q_n^+, D_q	$\mathbb{R}^{39}$; 39	Current/outcome physical observations and their dimension.
$a_n^*, \tilde{a}_n, a_n$	$[-1, 1]^4$ for valid commands	Desired, inverse-proposed, and executed commands.
c	Task identifier	One of the four manipulation task embeddings.
$d_n, \hat{d}_n$	$\mathbb{R}^{39}$	Standardized observed and predicted observation change.
$\mu_\Delta, \sigma_\Delta$	$\mathbb{R}^{39}$, fixed	Offline mean and positive scale of observation changes.
$\theta, \theta_0, g_\theta$	Trained parameters / predictor	Offline parameters; frozen deployment copy; endpoint predictor.
$h_n, \tilde{h}_n, d$	$\mathbb{R}^{96}$; 96	Original/corrected action feature and hidden width.
$\bar{h}, B, r, z_n$	$\mathbb{R}^{96}$; $\mathbb{R}^{30\times96}$; 30; $\mathbb{R}^{30}$	Fixed mean, fixed basis, basis rank, and compressed feature.
ρ, C_ρ	Route; $\mathbb{R}^{96\times30}$	Inferred mechanism and its online residual matrix.
$W_{\text{out}}, b_{\text{out}}$	$\mathbb{R}^{39\times96}$; $\mathbb{R}^{39}$	Frozen linear endpoint readout.
j, J, T_j, S_j	Index; 32; map; scalar	Candidate, bank size, command transform, and cumulative loss.
$k, g, \hat{k}, \hat{g}$	Scalars	Shear/gain and their selected candidate estimates.

Symbol	Shape or value	Meaning
η, R_C, δ	0.02, 0.20, 0.05	Matrix step length, matrix norm bound, command correction cap.
λ_b, λ_a	0.25, 4	Fixed binding/action loss weights in joint training.
E_M, G_M	MSE; percentage	Error of method M and gain relative to frozen prediction.
$\ell_m, \mathcal{L}_{\text{next}}$	Scalar losses	Per-stream and grouped sensory prediction objectives.
$\mathcal{L}_{\text{sem}}, \mathcal{L}_{\text{bind}}, \mathcal{L}_{\text{act}}$	Scalar losses	Semantic classification, image/text association, and action prediction objectives.
$\ell_{n,j}$	Scalar	Observed transition loss for candidate j.
L_n, A_n, C'	Scalar; $\mathbb{R}^{96\times 30}$; same	Matrix-write loss, its gradient, and tentative matrix before projection.
ϵ	10^{-12}	Safeguard for norm division in the matrix update.
e_n, v_n	$\mathbb{R}^{39}$	Uncorrected prediction residual and output-space correction.
N, y_i, p_{i,y_i}	Count; class; probability	Batch size, class label and its predicted probability.
u_i, v_j	$\mathbb{R}^{64}$, unit norm	Image and text binding vectors (distinct from membrane variables).
P_i, b_{ij}, τ_b	Index set; scalar; 0.12	Matching-class texts, image/text similarity and binding temperature.

A.2 Data interfaces and the shared core

The audio streams use 224 channels for SHD and 285 for SSC. SHD input is zero-padded to the shared audio stem's width, while the corresponding output heads retain their original dimensions. Retinal image and rendered-word streams each use 40 channels and share an input stem. The video stream uses 896 pooled feature channels. All sensory sequences have length 16. Retinal data come from the digit-image source in `sklearn.datasets.load_digits`; rendered words are represented by spatial features under a common scanpath. Video inputs are causal feature changes from a content-disjoint Something-Something-V2 cache. The study uses cached inputs with their recorded splits rather than collecting physical event-camera measurements.

The model has width 96, two shared blocks, four attention heads, and a feed-forward width of 192. Input stems apply layer normalization and linear projection, followed by position and modality embeddings. Input spikes are added to the projected numerical features. Each shared block has spike-based attention and a spiking feed-forward path, both with residual connections. The output is layer-normalized. Sensory next-slice prediction adds the learned output-head change to the current slice, so copying the current feature is the natural zero-change reference.

The attention implementation restricts each query to a local causal window of at most four positions, rather than selecting the four largest scores from all past positions. Queries, keys, and values are formed with LIF units; membrane-dependent gates modulate attention scores, which are normalized by softmax within the available window. This limits the number of attention pairs. We do not measure neuromorphic energy in these experiments. LIF state is initialized for each input sequence; deployment memory resides in the explicit external state rather than an unbounded cross-episode membrane history.

The action branch embeds the 39-coordinate observation and four-coordinate command into two tokens. Task and token-type embeddings are added to them, and the same core processes the pair. Its final token provides the action feature h_n for the linear readout. The complete checkpoint has 1,451,388 parameters. The joint-training configuration exposes 605,079 trainable parameters; the main deployment interface freezes the selected checkpoint.

A.3 Full offline objectives

Let ℓ_{SHD} and ℓ_{SSC} be Equation (2) on the two audio sources, and let $\ell_{\mathrm{image}}, \ell_{\mathrm{text}}, \ell_{\mathrm{video}}$ be the other sensory losses. The grouping used in training is

$$\mathcal{L}_{\mathrm{next}} = \frac{1}{4}\left[\frac{\ell_{\mathrm{SHD}} + \ell_{\mathrm{SSC}}}{2} + \ell_{\mathrm{image}} + \ell_{\mathrm{text}} + \ell_{\mathrm{video}}\right]. \tag{9}$$

Audio receives one group weight despite having two sources. The persistence denominator is floored at 10^{-8} in code to prevent division by zero. This equation defines the training loss, whose denominators come from the training split. Evaluation aggregation is specified separately in Appendix C.4.

For a batch of N labeled examples, let y_i be the class of example i and let p_{i,y_i} be its predicted softmax probability. Semantic cross-entropy is $-N^{-1}\sum_{i=1}^{N}\log p_{i,y_i}$, averaged across the five sensory streams. The coupled heads flatten the length-16 hidden sequence, normalize it, apply dropout during training, and predict the relevant classes. The class counts are 20 for SHD, 35 for SSC, 10 for retinal digits/words, and 42 for the video cache. An inherited stable semantic branch separately operates on standardized flattened sensory inputs. Both branches are frozen at deployment; the joint-learning semantic results concern the coupled branch.

For binding, N is the number of paired image/text examples in the batch. The subscripts I and T below denote image and text. Let u_i and v_j be unit-normalized 64-dimensional image and text representations, respectively. These are distinct from the membrane symbol u_τ in Equation (1). Define similarity $b_{ij} = u_i^\top v_j/\tau_b$, where the temperature is $\tau_b = 0.12$. Let P_i contain all text indices whose digit class matches image i. The image-to-text loss is

$$\mathcal{L}_{I\to T} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\sum_{j\in P_i}\exp(b_{ij})}{\sum_{j=1}^{N}\exp(b_{ij})}. \tag{10}$$

This assigns probability mass to all matching-class texts, rather than one arbitrarily chosen pair. The reverse loss $\mathcal{L}_{T\to I}$ exchanges image and text roles, and $\mathcal{L}_{\mathrm{bind}} = (\mathcal{L}_{I\to T} + \mathcal{L}_{T\to I})/2$. The binding representation averages hidden features across sequence positions before its projection.

The action objective is Equation (3). Its mean and scale are fit from offline observation changes and reused during deployment. The weights in Equation (4) are the fixed configuration used by the recorded study. Sequential action training runs for 800 updates. The additional joint stage uses 200 updates, batch size 32, and learning rate 10^{-4}. Sensory prediction and action prediction both supply gradients to the shared core in this joint stage.

B Online algorithm and state accounting

B.1 Initialization, routing, and state lifetime

The two residual matrices and all candidate scores start at zero. The stored counter starts at zero. At the start of each episode, the current route is source, the route is unlocked, and the temporary signature and

pending-loss lists are cleared. The matrices and cumulative scores persist across episodes within a run. A new run creates a new state.

The four routing classes are source, shear, attenuation, and noise. The router is a frozen two-layer network with a 64-unit ReLU hidden layer, fit offline using calibration signatures. Online, it receives the mean signature of the first four transitions. A signature contains candidate losses relative to identity, signed and absolute identity prediction errors, signed and absolute commands, and current physical observations. It also contains signed and absolute observation changes and cross-products between the first two command coordinates and the first six observation-change coordinates. These concatenate to 247 values per transition. The true mechanism label is used to fit the router offline but is absent from its online input.

Once the fourth signature arrives, the router locks for that episode. For shear or attenuation, the four pending loss vectors are added to the persistent scores, and their count is committed. Subsequent outcomes add one new loss vector each. Source and noise routes keep the identity command transformation and do not update residual matrices. Both temporary lists are cleared when the route locks. The scores are shared across tasks and indexed by candidate; the two matrices are indexed by mechanism family.

The selected candidate is the minimum cumulative score within the locked family. Dividing scores by the shared count would leave this ordering unchanged. Ties use the first minimum in the fixed bank order, matching the implementation's `argmin`. Scores are computed from the frozen predictor, before the residual-matrix correction.

B.2 Candidate transformations and command correction

The identity candidate is followed by 15 shear values $k \in \{0.05, 0.10, \ldots, 0.75\}$ and 16 attenuation values $g \in \{0.20, 0.25, \ldots, 0.95\}$. For command coordinates (a_x, a_y),

$$T_{\text{shear}}(a; k)_x = a_x, \qquad T_{\text{shear}}(a; k)_y = a_y + k a_x, \tag{11}$$

$$T_{\text{attenuation}}(a; g)_{x,y} = g\, a_{x,y}. \tag{12}$$

Other coordinates are unchanged. Transformed candidates are clipped to $[-1, 1]$ before querying the predictor. The inferred inverse uses $\tilde{a}_y = a^*_y - \hat{k} a^*_x$ for shear, or $\tilde{a}_{x,y} = a^*_{x,y} / \max(\hat{g}, 0.20)$ for attenuation. The hat denotes the selected bank value. The floor 0.20 prevents excessive division and coincides with the lowest attenuation candidate. The change from the desired command is then capped by Equation (6). If the resulting command exceeds the action bounds, the entire proposal is replaced by the desired command for that example.

B.3 Exact matrix write

The fixed basis B contains the first 30 right singular vectors of centered offline action hidden features. After observing d_n and committing the candidate-score update, the prediction used for the matrix write is recomputed with the selected candidate and the current matrix. Let C denote this active matrix, and let

$$L_n(C) = \frac{1}{D_q} \| W_{\text{out}}(h_n + C z_n) + b_{\text{out}} - d_n \|_2^2, \qquad A_n = \nabla_C L_n(C). \tag{13}$$

Here L_n is the current matrix-write loss, A_n is its matrix gradient, and all other quantities are held fixed for this write. The normalized step and projection are

$$C' = C - \eta \frac{A_n}{\max(\|A_n\|_F, \epsilon)}, \qquad C \leftarrow C' \min\left(1, \frac{R_C}{\max(\|C'\|_F, \epsilon)}\right), \quad \epsilon = 10^{-12}. \tag{14}$$

C' is the tentative matrix before projection, and ϵ safeguards division by a zero norm. The Frobenius norm $\| \cdot \|_F$ is the square root of the sum of squared matrix entries. Normalization controls step length; projection

bounds the matrix norm. A zero gradient yields a zero step. No optimizer moments are maintained. This recomputed training loss is distinct from the pre-update prediction error already recorded for evaluation.

A useful interpretation. Let $e_n = d_n - g_{\theta_0}(q_n, T_j(a_n), c)$ be the uncorrected error after candidate selection, and let $v_n = W_{\text{out}} C_\rho z_n$ be the correction in output space. Expanding the square gives

$$\frac{\|e_n\|_2^2 - \|e_n - v_n\|_2^2}{D_q} = \frac{2e_n^\top v_n - \|v_n\|_2^2}{D_q}. \tag{15}$$

The first term rewards alignment with the residual; the second penalizes an oversized correction. Thus prediction improvement depends on its direction as well as its magnitude. This identity concerns prediction error, while the separate inverse determines command changes.

B.4 One decision cycle

1. Read the current physical observation and desired policy command. Use the previously available route and scores to form a capped command.
2. Predict its outcome with the frozen core and the currently available residual matrix. Save this prediction for evaluation.
3. Execute the command and wait for the outcome. In live attenuation control, the actuator-transformed command is held over the macro horizon.
4. Score the saved prediction against the observed outcome before writing either external component.
5. Compute all candidate losses with the uncorrected frozen predictor. Before route lock, append the bounded signature/loss prefix. At the fourth outcome, infer the family and, for shear or attenuation, commit the four pending loss vectors exactly once. Clear the temporary prefix. On subsequent outcomes in these routes, add only the new loss vector.
6. If the route is now locked to shear or attenuation, perform one matrix write with Equation (14). The fourth outcome is the first eligible write; its candidate losses are not added a second time.
7. Continue with the next decision. The first opportunity for a changed command is decision five.

The code provides separate `predict`, `inverse`, and `observe` operations to keep these read/write times explicit.

B.5 Complete mutable-state ledger

The allocated tensor block is 24,384 bytes. The active entries use 23,172 bytes; the remaining entries preserve the historical allocation layout. The prefix is bounded at four observations and released on route lock. Python-object overhead and transient inference/gradient buffers are not included in this numeric-state ledger. The same persistent-state accounting includes trainable prediction parameters, optimizer tensors, and replay for the comparison arms. Bytes are not a hardware energy measurement.

Table 6: Peak mutable numeric allocation in FP32. Fixed model/router/basis parameters and transient activations are outside this adaptation-state metric.

Object	Arithmetic / lifetime	Bytes
Two residual matrices	$2 \times 96 \times 30 \times 4$; stream	23,040
Candidate scores and count	$(32 + 1) \times 4$; stream	132
Compatibility padding	303×4; fixed allocation	1,212
Routing signatures and losses	$4 \times (247 + 32) \times 4$; prefix	4,464
Peak total	Tensor block plus prefix	28,848

Table 7: Experimental units and the question answered by each study.

Study	Units	Question
Main stream	3 foundations, 3 action seeds each	Prediction, tracking, storage, return correction, latency
Routed actuator RLS	Same 3×3 layout, separate replay	Actuator-space identification under the six-stage protocol
Component roles	5 model-seed clusters	Matrix contribution versus score-only and no-inverse variants
Behavioral equivalence	4 controllers, 3 tasks, 3 environment seeds	Bounded versus legacy history; first changed command
Closed loop	5 models, 3 tasks, 5 replicas per arm	Reward and success under attenuation
Joint learning	5 action seeds, one sensory foundation	Shared-core predictive and semantic learning
Training-stage pilot	One foundation; matched cached stream	Same adaptation interface across offline stages
Controlled probes	3 seeds per separate model study	Temporal, future-statistic, and factor-association tests
Output calibration	One foundation; 4 paired units per arm	Hidden versus output-space correction

C Protocols and extended experimental evidence

C.1 Experimental units and statistics

Table 7 separates the study units. A foundation is a sensory checkpoint; an action seed initializes or trains its action path. Multiple action seeds under one foundation are not treated as independent foundations.

For the main stream, each action-seed run first averages the measured shifted transition errors, then computes its gain relative to the paired frozen arm. There are 126 measured shifted transitions per run, after omitting each episode's four-transition prefix from the score. All methods replay the same observations; tracking is evaluated through the known actuator map. Action seeds are averaged within each foundation, and the reported mean averages foundations. The archived 95% intervals use 20,000 bootstrap draws of foundation means. Update latency uses 2,988 synchronized samples per adaptive arm after discarding 16 warm-up updates from each run.

Source stages reuse router-calibration episodes and four-transition prefixes; the provenance record identifies 48 reused source-prefix rows per evaluated replica. Shifted stages use a separate execution archive. This defines the calibrated-source reentry measurement in the main table. The sequence can retain matrices

Table 8: Prediction gain by independent foundation (%). Each row averages three action seeds.

Foundation	External state	Adapter	Prediction-path	Replay
33401	21.80	14.60	25.30	25.08
33402	20.34	17.42	17.47	16.76
33403	15.71	14.82	22.79	19.29

and scores across stages while re-establishing the route at the start of each episode.

Closed-loop statistics preserve method pairing while resampling model, task, and replica axes independently, with 20,000 draws. Success is the episode-level indicator of task completion; cumulative reward sums environment rewards. The reference code's older control runner has its own flat-episode bootstrap; the released recorded results contain the multi-axis intervals reported here.

C.2 Independent foundation results

The three foundation rows show how prediction gain varies across independently trained foundations. The frozen reference has zero gain by definition. The main result is a mean trade-off over these independently trained foundations; all rows and comparison arms are retained in the evidence files.

The same-protocol routed actuator RLS uses 4,112 bytes, including its routing prefix. It obtains 0.58% prediction gain and -1.05% tracking gain; the replicated external-state arm obtains 19.72% and 3.87%, respectively. Both return zero correction on calibrated source revisits. This RLS estimates actuator coupling and is distinct from the output-space calibrators below.

C.3 Controlled temporal and cross-modal probes

These historical studies use separate event encoders, with predictive and contrastive objectives, and are not additional main-stream foundations. Identity metadata builds paired views; the future-prediction targets themselves are computed from observations.

In the temporal probe, the full sequence is compared with two controls. Order shuffling permutes non-time event features across fixed timestamps. The rate-grid control sorts event features and assigns evenly spaced timestamps. The outcome is future-slice negative log-likelihood (NLL), where lower values indicate better prediction. These controls test sensitivity to the temporal structure of this task.

The future-statistic probe predicts normalized histograms relating event time and feature coordinates in later image/audio views. A learned encoder is compared with a random encoder using the same prediction task. The composition probe receives factor segments and compares true image/audio factor associations with shuffled associations. Its score is top-1 factor association accuracy. Table 9 reports the archived three-seed means; the separate objectives have different metric scales.

C.4 Three-stage joint-learning results

Table 10 reports all three stages of the five-action-seed study. Its sensory metric averages four groups: audio (the mean of SHD and SSC), image, text, and video. The stored normalized values are used as recorded. The reference evaluator separately uses validation-set persistence denominators and an unweighted five-stream mean; that evaluator's aggregate should not be substituted for this archived four-group statistic. Relative to action-only updates, joint training increases action MSE by 3.30% on average across paired seeds (95% CI:

Table 9: Controlled event-model probes, three-seed means. Lower NLL/loss and higher accuracy are better. Temporal, future-statistic and factor-association studies correspond to archived M60, M69 and M71 records.

Study	Condition	Metric	Mean
Temporal	Ordered sequence	NLL	4.9118
Temporal	Rate-grid control	NLL	6.9293
Temporal	Shuffled order	NLL	11.0357
Future statistics	Learned encoder	Loss	0.1103
Future statistics	Random encoder	Loss	0.1397
Factor association	True pairing	Accuracy (%)	46.6667
Factor association	Shuffled pairing	Accuracy (%)	6.6667

Table 10: All stages of the five-action-seed joint-learning study. Sensory MSE is the archived four-group normalized mean; accuracy and retrieval are percentages.

Stage	Action MSE	Sensory MSE	Semantic (%)	Retrieval (%)
Sequential	0.1463	0.6776	19.1488	10.0000
Action-only shared update	0.1174	0.6786	19.2655	8.5000
Joint shared update	0.1212	0.6725	20.9440	11.8125

1.65%–4.94%), while improving class retrieval by 3.31 percentage points (95% CI: 0.94–5.38). Joint updates improve retrieval, while action-only updates favor action prediction.

C.5 Training-stage comparison

The pilot begins with one common sensory-pretrained checkpoint. From the sequential action-trained model, three copies receive 200 additional updates: joint training, action loss alone, or joint training without the sensory next-slice term. Thus the no-next variant removes the term in the final stage, not from the full pretraining history. Offline fitting supplies each model's basis and router using the same procedure and hyperparameters.

All models replay the same cached six-stage trajectory and compare frozen, score-only, and full external state on 126 shifted transitions after routing. Table 11 shows that the interface can be used across these training stages. The ordering of adapted errors differs from the ordering of offline action errors. Lower offline action error alone does not determine which checkpoint has the lowest error after adaptation in this pilot.

Table 11: One-foundation training-stage pilot. MSE is evaluated on 126 shifted transitions after the observation prefixes. All variants inherit sensory predictive pretraining.

Training stage	Frozen MSE	Score MSE	Full MSE	Gain (%)
Sequential	0.3927	0.3661	0.2878	26.72
Action-only	0.4408	0.3797	0.2918	33.81
Joint	0.4097	0.3992	0.3116	23.96
Joint, no-next	0.4270	0.4129	0.3186	25.39

Table 12: Separate output-calibration development protocol: one sensory foundation, four paired units per method. These results are not pooled with the main-stream comparison.

Correction	MSE	Gain (%)	Bytes	Update (ms)
Hidden residual state	0.2640	23.34	28,848	11.70
Output NLMS	0.1652	51.70	13,956	11.03
Output RLS	0.1839	45.84	21,156	11.10
Low-rank readout	0.2961	14.14	30,532	11.97

C.6 Direct output-space calibration

The additional development comparison applies corrections at the output rather than the hidden feature. Table 12 preserves all methods: hidden residual state, direct-output NLMS, direct-output RLS, and a low-rank readout update. It uses one sensory foundation and four paired units per method, rather than the three-foundation main comparison.

In this setting, direct-output NLMS and RLS achieve larger prediction gains with less mutable state than the hidden-state correction. This comparison favors output-space correction in this development setting. We report it separately from the main-stream frontier because the protocols differ. The released records include every method arm with its protocol.

C.7 Closed-loop detail

Each model/task controller is initialized once, then processes five fresh trajectory replicas in order. External state resets its route at each episode but keeps its matrices and candidate scores. Scalar RLS and gain-memory replay also retain their accumulated state across these replicas. All episodes use attenuation 0.4 and last 13 decisions, or 104 simulator steps.

The live-control tuning arm uses AdamW at learning rate 10^{-5} on parameters whose names do not contain `semantic`. Gain-memory replay retrieves local gain estimates from observation/task features using nearest neighbors, with capacity 141. It does not train the prediction path with replay as the main-stream Adam arm does. The scalar RLS controller accumulates a numerator and denominator using finite-difference model sensitivity. The oracle divides by the true attenuation and clips to the valid command range; unlike the external-state controller, it has no 0.05 correction cap. Consequently, the main-stream storage and latency entries are not assigned to these live-control configurations.

All six arms appear in Table 3, including the scalar RLS and oracle references. The external-state reward gains by task are 2.31 for pick-place, 11.61 for push, and 9.79 for reach. The five model-seed gains range from 7.19 to 8.66. Prediction-model tuning and external state both achieve 66.67% aggregate success. The scalar RLS obtains the largest reward among the non-oracle methods in this single-fault control protocol. External state improves reward over frozen control, while scalar RLS is the stronger controller for this particular fault.

D Reproduction guide and release contents

The code is organized around the model, adaptation interface, reference experiments, and recorded evidence. The repository is `https://github.com/Oooorca/SpikeWorld`. The release contains the paper source, figures, numerical tables, and verification tools. The entry points below cover recorded-result verification, reference model execution, and fresh experiments.

Recorded-result verification. `python tools/verify_current.py` checks the exported evidence hashes and reconstructs the main-stream results from nine run records. It recomputes prediction gain, tracking gain, source-return correction, state sizes, and latency medians. Other archived files are integrity-checked by that tool. The additional verifier, `tools/verify_preprint.py`, and the numerical figure generator trace table entries to named JSON fields and recompute closed-loop task differences. These operations use stored experimental outputs; they do not train models.

Executable reference interfaces. The package's tests exercise prediction shapes, causal attention, routing, memory bounds, action correction, and update timing. A representative historical checkpoint is identified by the artifact manifest and distributed as a hash-checked release asset. It is separate from the three independent foundations in the main-stream table. The entry points `spikeworld-joint`, `spikeworld-adapt`, `spikeworld-control`, and `spikeworld-audit` expose the reference training, adaptation, control, and inspection workflows.

Inputs for fresh experiments. Fresh reference runs require sensory caches, action fitting and evaluation arrays, trained checkpoints, and fitted router/PCA bundles. The released artifact guide lists these fields and the available checkpoint/shift-stream assets. Several full-training caches, all current foundation checkpoints, and the later study executors are not bundled. Consequently, the released record verifier and reference interface tests are immediately executable, while a fresh reproduction of every current experiment additionally requires the listed upstream artifacts. Dataset access and its terms remain with the original sources.

Table 13: Code and result map. Paths are relative to the repository root.

Path	Contents
`src/spikeworld/model.py`	Input adapters, LIF units, shared core, semantic paths, endpoint head.
`src/spikeworld/training.py`	Sensory, semantic, binding and action objectives; training routines.
`src/spikeworld/adaptation.py`	Candidate transforms, frozen router, scores, matrix update and byte ledger.
`results/current/`	Current manuscript records, including main stream, component study, closed loop and joint learning.
`results/current/probe_records/`	Underlying controlled event-model probe outputs.
`results/current/additional_protocol/`	Separate direct-output calibration comparison, all method arms preserved.
`results/raw/`	Preserved legacy result snapshot, with its own verifier.
`artifacts/`	Data/checkpoint requirements, online field contracts and download hashes.
`paper/`	This preprint, editable figures and LaTeX sources.

The supported Python version for the package is 3.11. The lockfile records dependencies for optional training, control, and development extras. Simulation requires Meta-World and MuJoCo; the recorded-result verifier does not. Hardware latencies in the paper are the archived synchronized GPU measurements, rather than timings of the present CPU packaging checks.

AI assistance

Generative AI tools assisted with literature lookup, research discussion, manuscript drafting and language editing, figure preparation, and preparation of the accompanying verification and release code. The authors retain responsibility for the scientific claims, experimental records, and released artifacts. The experimental quantities reported here come from the identified result records; this editorial revision does not introduce new training runs.